

A Project Report
On
“Image extraction and categorisation using Python and NLP”

Submitted for the fulfilment of the requirement

*for the Summer Internship program in
Computer Science at
ONGC*



Compiled by:

Ashis Prajapati

Under the guidance of:

Mr. Sanjay Kumar Arora
General Manager (Programming)
EDS Dehradun



DECLARATION

I hereby declare that the work presented in this project report entitled “**Image extraction and categorisation using Python and NLP**” in partial fulfilment of requirement for the Award of Degree of Bachelor of Technology, submitted in the Department of Computer Science, Graphic Era Deemed University is an authentic record of my work carried out during the Industrial Training from 7 June 2024 to 7 August 2024, under the guidance of **Mr. Sanjay Kumar Arora General Manager (Programming) EDS Dehradun.**

Sign

(Ashis Prajapati)



CERTIFICATE

This is to certify that Ms. Ashis Prajapati, a student of BTech (C.S.E AI & ML) of SRM Institute of Science & Technology has done his summer internship program/Industrial Training at ONGC Dehradun. The project work entitled “Image extraction and categorisation using Python and NLP” embodies the original work done by Mr Ashis Prajapati during his two-month Industrial Training period from 7th June 2024 to 7th August 2024 .

Mr. Sanjay Kumar Arora
General Manager (Programming)
EDS Dehradun



ACKNOWLEDGEMENT

My internship with Oil and Natural Gas Corporation Ltd. was a profound learning experience and a period of significant professional growth. I feel exceptionally fortunate to have had the privilege to be part of such an extraordinary opportunity, which has left an indelible mark on both my personal and professional journey.

I am deeply grateful to Mr. Sanjay Kumar Arora, GM (Programming), who graciously served as my mentor during this transformative phase. His insightful advice and unwavering support illuminated my path throughout this immersive journey.

I extend my special gratitude and thanks to Ms. Somya Sangal, Programming Officer at ONGC, for her exceptional communication and guidance throughout our training. Her unwavering support, patience, and willingness to help us by clearing our doubts was invaluable and greatly appreciated.

This internship represents a crucial step in my career, equipping me with invaluable skills and knowledge of immense potential. I am wholeheartedly committed to applying these assets effectively and am determined to achieve my career aspirations through continuous growth. In conclusion, I am profoundly grateful to my parents for their unwavering support, which has been my driving force throughout this endeavour.

Sincerely
Ashis Prajapati

CONTENTS

Introduction: About ONGC	6
ABOUT Exploration data services	
An introduction	9
Software	9
Database centres & data flow	10
In-house data management software	11
PROJECT	
Abstract, objective,scope and methodology	13
Software Requirement	15
Functionalities and Database	16
Source Code	17
Output	26
Conclusion	27
Bibliography	28

INTRODUCTION

Oil and Natural Gas Corporation Limited (ONGC) is a major Indian multinational oil and gas company headquartered in Dehradun, Uttarakhand, India. Established in 1956, ONGC is a flagship organization under the Ministry of Petroleum and Natural Gas of the Government of India.

ONGC is engaged in the exploration, production, and refining of oil and gas, making it one of the largest publicly-traded oil and gas companies in India. The company's activities span the entire hydrocarbon value chain, from exploration to production, refining, and distribution. ONGC plays a vital role in ensuring the energy security of India by contributing significantly to the country's domestic oil and gas production.

HISTORY

Established in the late 1950s, the Oil and Natural Gas Corporation Limited (ONGC) has carved a storied history as a vital pillar of India's energy landscape. Born from the vision of post independence self-sufficiency, ONGC was founded in 1955 to spearhead the exploration and production of indigenous oil and gas resources. Rapidly making its mark, ONGC achieved its first significant oil discovery in 1960 and extended its reach to offshore drilling, discovering the Bombay High oil field. The subsequent decades witnessed ONGC's evolution into a full fledged corporation, marked by its forays into international ventures, adoption of advanced technologies, and integration of various oil-related entities. Amidst growing environmental awareness, ONGC embraced sustainable practices and ventured into renewable energy. Today, its journey stands as a testament to its enduring commitment to powering India's progress, innovating for a sustainable future, and maintaining a pivotal role in the nation's economic and energy security endeavors.

VISION AND MISSION

VISION

To be global leader in integrated energy business through sustainable growth, knowledge excellence and exemplary governance practices.

MISSION

World Class

- Dedicated to excellence by leveraging competitive advantages in R&D and technology with involved people.
- Imbibe high standards of business ethics and organizational values.
- Abiding commitment to safety, health and environment to enrich quality of community life.
- Foster a culture of trust, openness and mutual concern to make working a stimulating and challenging experience for our people.

- Strive for customer delight through quality products and services. **Integrated In Energy Business**
- Focus on domestic and international oil and gas exploration and production business opportunities.
- Provide value linkages in other sectors of energy business.
- Create growth opportunities and maximize shareholder value.

FRONTIERS OF TECHNOLOGY

- **Seismic Excellence:** ONGC excels in seismic data handling, using advanced acquisition, processing, and interpretation facilities.
- **Virtual Reality Prowess:** Featuring a world-ranking Virtual Reality Interpretation setup, ONGC is committed to immersive data analysis.
- **Global Partnerships:** Collaborations with major industry players like Transocean, Schlumberger, and more, emphasize ONGC's global impact.
- **International Reach:** Alliances with Petrobras, Norsk, ENI, and Shell highlight ONGC's significance in the global energy sector.
- **Efficiency Emphasis:** ONGC's extensive ERP implementation showcases its dedication to operational efficiency and effective management.

GLOBAL RANKING

- **Top Energy Company in India:** Oil and Natural Gas Corporation (ONGC) ranked as the best energy company in India in the prestigious Platts Top 250 Global Energy Company Rankings 2014.
- **Global Recognition:** ONGC improved its global ranking, securing the 21st position among global energy majors, and climbed to the 5th spot in the Asia/Pacific (APAC) region, up from 7th the previous year.
- **Exploration and Production Leadership:** ONGC maintained its 3rd position as a global Exploration and Production company, further solidifying its industry leadership.
- **Fast-Lane Initiatives:** ONGC's upcoming initiatives and accelerated production plans are set to contribute to even greater milestones in the future.
- **BT 500 Ranking:** ONGC's recent rise to Number 2 among Indian corporate giants in the BT 500 ranking adds to its string of achievements.
- **Asian Influence:** ONGC's ranking at 5 in the APAC region reflects the growing influence of Asian companies in the global energy sector.

- **Indian Companies in the List:** Thirteen Indian companies, including Reliance Industries Ltd, Indian Oil Corporation, and NTPC, are featured in the Top 250 Global • Energy Company list by Platts.
- **Performance Metrics:** The Platts survey evaluates companies based on key financial metrics such as asset worth, revenues, profits, and return on invested capital. All companies in the list have assets exceeding US \$5 billion.

EXPLORATION DATA SERVICES

About Exploration data services –An Introduction

During past five decades of extensive hydrocarbon exploration over a vast geographical area covering almost all the sedimentary basins in India, ONGC has acquired enormous volume of geological, geophysical and engineering data. In line with the global upstream oil majors, underlining the importance of computerized data archival, ONGC also initiated data management activities way back in 1976. A well information system on IBM-370 at

KDMIPE was developed “in-house”. The data management activities got a boost when VAX 3400 computer system along with RDBMS software was commissioned at KDMIPE in 1991. Furthermore, other isolated systems also emerged at different work centers of ONGC to cater to the data management needs. These in-house database systems were developed using different types of formats and platforms. Eventually, in-house database systems were felt inadequate and required to be replaced with system matching international standards in the industry. ONGC, in late 1990's decided to implement the recent advancements of the web based E&P information technology into its upcoming data management systems. The new implementation plan aimed to make available all information pertaining to an area, including the work previously done by archiving historical analysis, to business users.

In the aforesaid background, ONGC decided to implement **Exploration and Production Information Network (Exploration data services)** project comprising of people, processes, tools, data and a hierarchy of corporate, regional and working project database in a phased manner. The project was intended to establish an organization wide dynamic, "Virtual" database having GIS features and Web 2 capabilities, to interconnect different data stores located at geographically diversified locations. Having taken shape according to the vision, intention and need of various work-centers of ONGC, today, Exploration data services is ONGC's full-fledged technical data repository housing all technical data generated throughout ONGC. This includes WCRs, Log data, Seismic data, Geology Data, Lab data, Drilling Data and Production Data.

SOFTWARE: FINDER

Until 2013, Exploration data services housed all its data using software called “Finder”. Finder is a product of Schlumberger Information Systems (SIS). Finder uses Oracle Database and is built to run on the old Sun Operating system. A Sun server is used to host the Finder software. Below is a screenshot of Finder user interface:



THREE-TIER SETUP

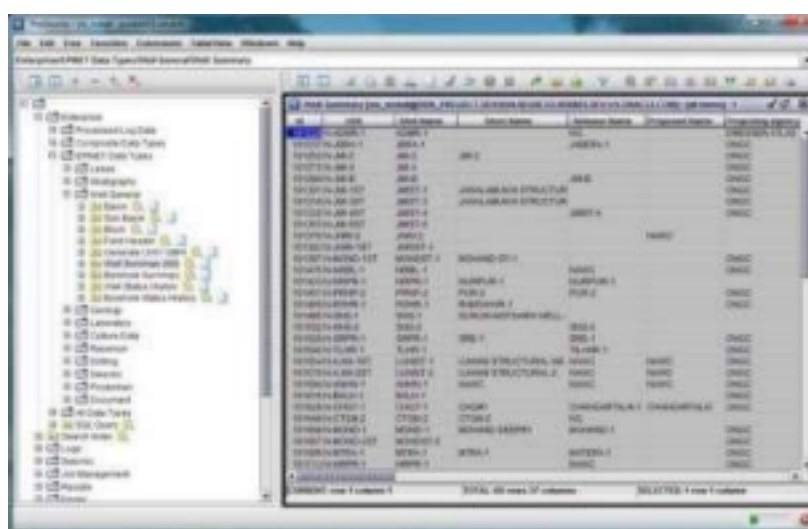
When Finder was in use, Exploration data services was organized in a three-tier model with secondary and tertiary level data-centers spread across each basin. The following Diagram shows the old three-tier setup:

9



Three-tier setup of Exploration data services under Finder

ProSource After 2013, all data in Exploration data services was managed through a software package called “ProSource”. ProSource has been in full use only since December 2013. It is also a product of Schlumberger Information Systems (SIS). ProSource uses Oracle Database to manage data and is built on the seabed Data Model. The User Interface of ProSource is web based and is built on Java. ProSource runs on Red Hat Enterprise Linux 5.5. A product of Oracle called “Golden Gate” is used to replicate data from all centers into the database at Dehradun Corporate center. Migration from Finder to the new ProSource system took place in several phases, covering one region at a time. The final phase completed in December 2013 with the migration of data at Corporate Exploration data services center, Dehradun. Below is a screenshot of the new ProSource interface:

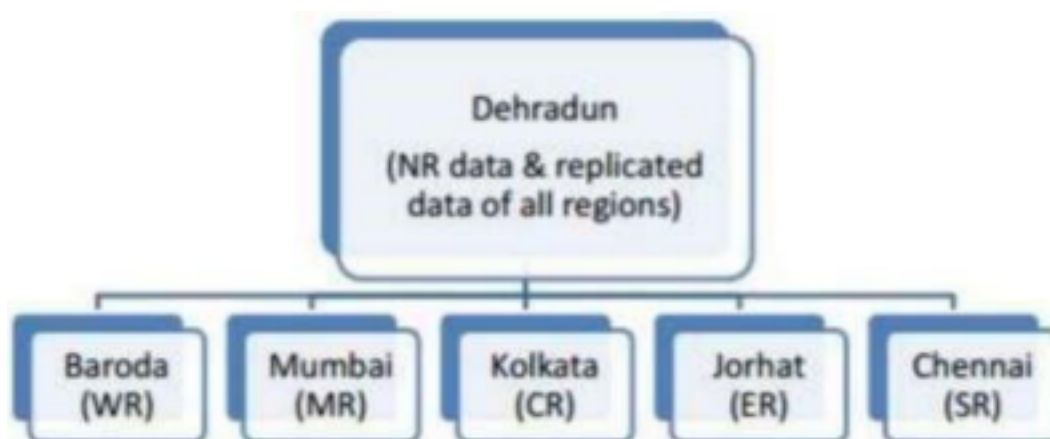


DATABASE CENTERS & DATA FLOW

Under ProSource, Exploration data services was organized into six data centers in a two-tier model with the corporate centre housed at KDMIPE, Dehradun. The data centers and the regions covered are as follows:

Data Center Location	Basin / Region covered	Region
Dehradun	Frontier Basin	Northern Region (NR)
Baroda	Western Onshore Basin	Western Region (WR)
Mumbai	Western Offshore	Mumbai Region (MR)
Kolkata	MBA Basin	Central Region (CR)
Jorhat	A&AA Basin	Eastern Region (ER)
Chennai	KG-PG Basin	Southern Region (SR)

Data from respective regions are populated and maintained at the respective centers. Data from all centers are centrally replicated at the Corporate Exploration data services Center, Dehradun via Golden Gate, a plugin of Oracle. The following diagram shows the setup:



Two-tier setup of Exploration data services under ProSource

DEVELOPMENT OF IN-HOUSE DATA MANAGEMENT SOFTWARE

During November 2012, it was decided that efforts would be made to maintain Exploration data services system using in-house talent. Eventually, in-house maintenance began in October 2014, saving ₹12 Crore annually since then. Leveraging in-house talent across all Exploration data services sites and collaborating over a period of 2 years, the Beta version of Exploration data services Portal & Loaders were ready and was rolled out exclusively for in-house testing by Exploration data services members, at the Annual Coordination Meeting held at Exploration data services Chennai. After extensive testing and improvements, in July 2017, the tested and improved version of In-house Exploration data services Software is rolled out across ONGC by Director (Exploration), Sh. A K Dwivedi via video-conferencing and was jointly witnessed by various offices across ONGC joining the live video conference. This In-house software is currently in use, with new features being added consistently.



EPINET now known as Exploration Data Services

Project

Image extraction and categorisation using Python and NLP

ABSTRACT:

This project, “**Image extraction and categorisation using Python and NLP**” focuses on developing a robust system for automatically extracting text and images from complex documents and classifying these images into categories using Natural Language Processing (NLP) techniques. This project leverages the power of Python programming, coupled with advanced NLP frameworks such as spaCy, to process and analyze text data embedded in images. This symbolises the new age approach at ONGC where technology is being embraced and applied for maximum efficiency and growth.

OBJECTIVE:

The project aims to classify various images in categories using NLP . The document used for this project is a well completion report (WCR) and the objective is to classify the various images into the defined categories using the text extracted from around the text as means to know what kind of image it is.

SCOPE:

The project scope involves developing an automated system for extracting and categorising images within well completion reports using Python and Natural Language Processing (NLP). This encompasses identifying the right , usable images extracting relevant texts from around them and preprocessing those texts then classifying them.

METHODOLOGY

1. Image Extraction

- Open PDF Document: Use the `fitz` library to open the PDF document.
- Iterate Through Page: Loop through each page in the PDF.
- Identify Images: Locate images on each page that meet size criteria (between 250x250 and 2000x2000 pixels).
- Extract Images: Extract image data and associated text (above and below the image) from each identified image.
- Save Images: Save the extracted images to the output directory with a filename based on the preprocessed text.

2. Text Preprocessing

- Load spaCy Model : Load the spaCy model (`'en_core_web_md'`) for text preprocessing.
- Clean Text : Remove non-alphanumeric characters and extra spaces from the extracted text.
- Preprocess Text : Convert text to lowercase, remove stop words, punctuation, and lemmatize the text using spaCy.
- Save Preprocessed Text : Write the preprocessed text to a file (`'extracted_names.txt'`).

3. Clustering Headings

- Load Preprocessed Text : Read preprocessed text from `'extracted_names.txt'`.
- Vectorize Text : Use `'TfidfVectorizer'` to convert text into numerical features.
- Cluster Text : Apply KMeans clustering algorithm to group similar text headings.
- Save Clustering Results : Save the clustering results to a CSV file (`'clustered_headings.csv'`).

4. Classifying Headings

- Load Classification Model : Use the `'transformers'` library to load a zero-shot classification model (`'facebook/bart-large-mnli'`).
- Define Categories : Define the categories for classification (e.g., "similarity factor", "types of images", "location map", etc.).
- Classify Headings : Classify each preprocessed text into one of the defined categories.
- Save Classification Results : Save the classification results to a CSV file (`'classified_headings.csv'`).

5. Organising Images by Category

- Load Image Data : Load the extracted image data from `'extracted_data.json'`.
- Load Classification Data : Load the classification results from `'classified_headings.csv'`.
- Match Images with Categories : Match each image with its corresponding classified category based on the preprocessed text.
- Create Category Directories : Create directories for each category.
- Move Images : Move images to their respective category directories. If no category is matched, move the image to an 'unclassified' directory.

TECHNICAL REQUIREMENTS:

<u>Libraries</u>	fitz ,Pillow,spacy ,scikit-learn,transformers,pandas,torch
<u>Python Version</u>	Python 3.7 or higher installed on your system.
<u>Files</u>	Ensure the original pdf file is in the same folder as well as other files being stored in a similar way.

TECHNOLOGIES USED:

Project : Image Extraction and Categorization Using Python and NLP

Image Processing and Extraction: PyMuPDF (fitz)

PyMuPDF (fitz) is used for extracting images and text from PDF files. It provides a powerful interface for working with PDF documents, allowing for precise extraction of images and associated text content. Pillow, a fork of the Python Imaging Library (PIL), is employed for image processing tasks, including opening, manipulating, and saving image files in various formats.

Natural Language Processing: spaCy

spaCy, an advanced NLP library in Python, is used for text preprocessing tasks. It provides functionalities such as tokenization, lemmatization, part-of-speech tagging, and named entity recognition, enabling efficient text analysis and manipulation. In this project, spaCy helps clean and preprocess text extracted from images to prepare it for further classification and clustering.

Machine Learning: scikit-learn

scikit-learn is a robust machine learning library in Python. It is used for implementing clustering algorithms, specifically KMeans, to group similar text headings. The library's tools for preprocessing, model selection, and evaluation are crucial for developing and fine-tuning the clustering models.

Text Classification: Transformers (Hugging Face)

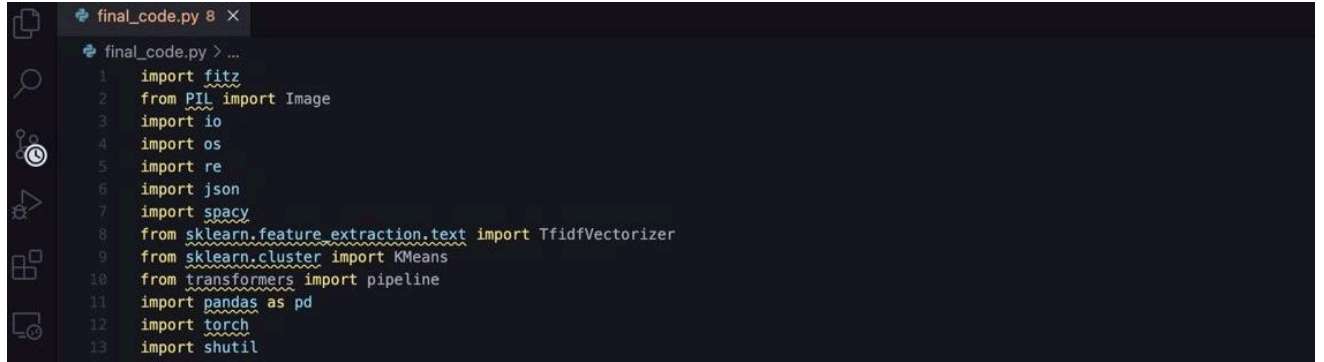
The `transformers` library by Hugging Face is employed for zero-shot classification. It allows the use of pre-trained models like BART (Bidirectional and Auto-Regressive Transformers) for classifying text into predefined categories without requiring task-specific training data. This capability is pivotal for accurately categorizing the extracted text headings.

File Management: shutil

The `shutil` module in Python is used for high-level file operations, such as copying and moving files. It plays a crucial role in organizing images into category-based directories after classification, ensuring a well-structured file system.

SOURCE CODE:

LIBRARIES USED :



```
final_code.py 8 X
final_code.py > ...
1  import fitz
2  from PIL import Image
3  import io
4  import os
5  import re
6  import json
7  import spacy
8  from sklearn.feature_extraction.text import TfidfVectorizer
9  from sklearn.cluster import KMeans
10 from transformers import pipeline
11 import pandas as pd
12 import torch
13 import shutil
```

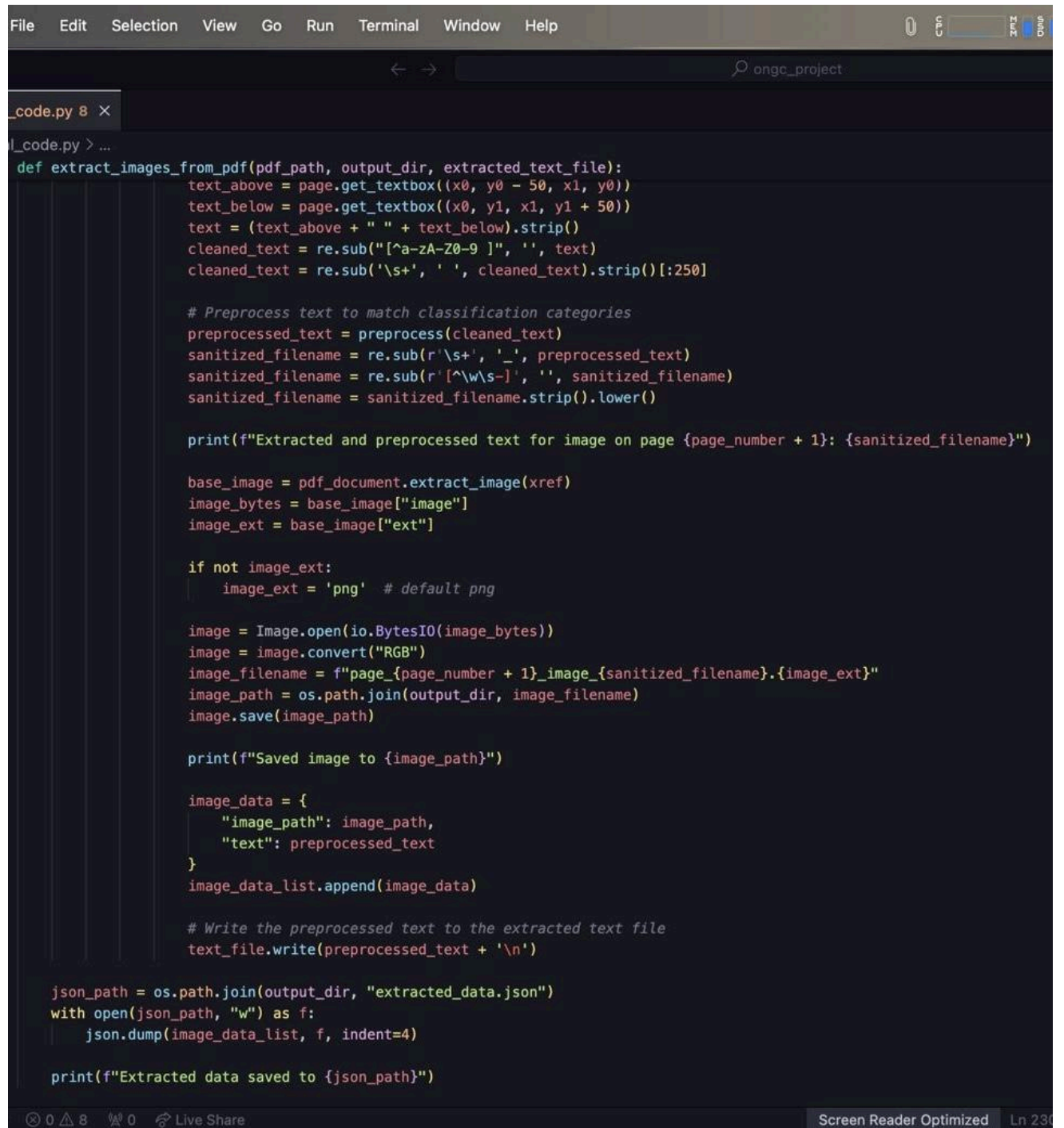
IMAGE & TEXT EXTRACTION :

```

final_code.py > ...
1  import fitz
2  from PIL import Image
3  import io
4  import os
5  import re
6  import json
7  import spacy
8  from sklearn.feature_extraction.text import TfidfVectorizer
9  from sklearn.cluster import KMeans
10 from transformers import pipeline
11 import pandas as pd
12 import torch
13 import shutil
14
15 def extract_images_from_pdf(pdf_path, output_dir, extracted_text_file):
16     """Extract images from a PDF and save them with corresponding text."""
17     pdf_document = fitz.open(pdf_path)
18     image_data_list = []
19
20     with open(extracted_text_file, 'w') as text_file:
21         for page_number in range(len(pdf_document)):
22             page = pdf_document.load_page(page_number)
23
24             for img in page.get_images(full=True):
25                 if 250 <= img[2] <= 2000 and 250 <= img[3] <= 2000:
26                     xref = img[0]
27                     image_bbox = page.get_image_bbox(img)
28                     x0, y0, x1, y1 = image_bbox
29
30                     text_above = page.get_textbox((x0, y0 - 50, x1, y0))
31                     text_below = page.get_textbox((x0, y1, x1, y1 + 50))
32                     text = (text_above + " " + text_below).strip()
33                     cleaned_text = re.sub("[a-zA-Z0-9 ]", '', text)
34                     cleaned_text = re.sub('\s+', ' ', cleaned_text).strip()[:250]
35
36                     # Preprocess text to match classification categories
37                     preprocessed_text = preprocess(cleaned_text)
38                     sanitized_filename = re.sub(r'\s+', '_', preprocessed_text)
39                     sanitized_filename = re.sub(r'^\w\s-', '', sanitized_filename)
40                     sanitized_filename = sanitized_filename.strip().lower()
41
42                     print(f"Extracted and preprocessed text for image on page {page_number + 1}: {sanitized_filename}")
43
44                     base_image = pdf_document.extract_image(xref)
45                     image_bytes = base_image["image"]

```

ashish 0 0 8 0 Live Share Screen Reader Optimized Ln 230, Col 1 Spac



```
File Edit Selection View Go Run Terminal Window Help
ongc_project

_code.py 8 x
_code.py > ...

def extract_images_from_pdf(pdf_path, output_dir, extracted_text_file):
    text_above = page.get_textbox((x0, y0 - 50, x1, y0))
    text_below = page.get_textbox((x0, y1, x1, y1 + 50))
    text = (text_above + " " + text_below).strip()
    cleaned_text = re.sub("[^a-zA-Z0-9 ]", '', text)
    cleaned_text = re.sub('\s+', ' ', cleaned_text).strip()[:250]

    # Preprocess text to match classification categories
    preprocessed_text = preprocess(cleaned_text)
    sanitized_filename = re.sub(r'\s+', '_', preprocessed_text)
    sanitized_filename = re.sub(r'[\w\s-]', '', sanitized_filename)
    sanitized_filename = sanitized_filename.strip().lower()

    print(f"Extracted and preprocessed text for image on page {page_number + 1}: {sanitized_filename}")

    base_image = pdf_document.extract_image(xref)
    image_bytes = base_image["image"]
    image_ext = base_image["ext"]

    if not image_ext:
        image_ext = 'png' # default png

    image = Image.open(io.BytesIO(image_bytes))
    image = image.convert("RGB")
    image_filename = f"page_{page_number + 1}_image_{sanitized_filename}.{image_ext}"
    image_path = os.path.join(output_dir, image_filename)
    image.save(image_path)

    print(f"Saved image to {image_path}")

    image_data = {
        "image_path": image_path,
        "text": preprocessed_text
    }
    image_data_list.append(image_data)

    # Write the preprocessed text to the extracted text file
    text_file.write(preprocessed_text + '\n')

    json_path = os.path.join(output_dir, "extracted_data.json")
    with open(json_path, "w") as f:
        json.dump(image_data_list, f, indent=4)

    print(f"Extracted data saved to {json_path}")
```

0 8 0 Live Share Screen Reader Optimized Ln 230

TEXT PREPROCESSING:

```
74 def preprocess(text):
75     """Preprocess text by removing stop words and punctuation, and lemmatizing."""
76     doc = nlp(text.lower())
77     return ' '.join([token.lemma_ for token in doc if not token.is_stop and not token.is_punct])
78
79 def preprocess_file(input_file_path, output_file_path):
80     """Read the original file, process the text, and write back the processed text."""
81     with open(input_file_path, 'r') as file:
82         lines = file.readlines()
83
84     processed_lines = [preprocess(line.strip()) for line in lines]
85
86     with open(output_file_path, 'w') as file:
87         for processed_line in processed_lines:
88             file.write(processed_line + '\n')
89
90     for processed_line in processed_lines[:10]:
91         print(processed_line)
92
93 > def cluster_headings(input_file_path, output_file_path, num_clusters=10):--
111
```

CLUSTER HEADINGS:

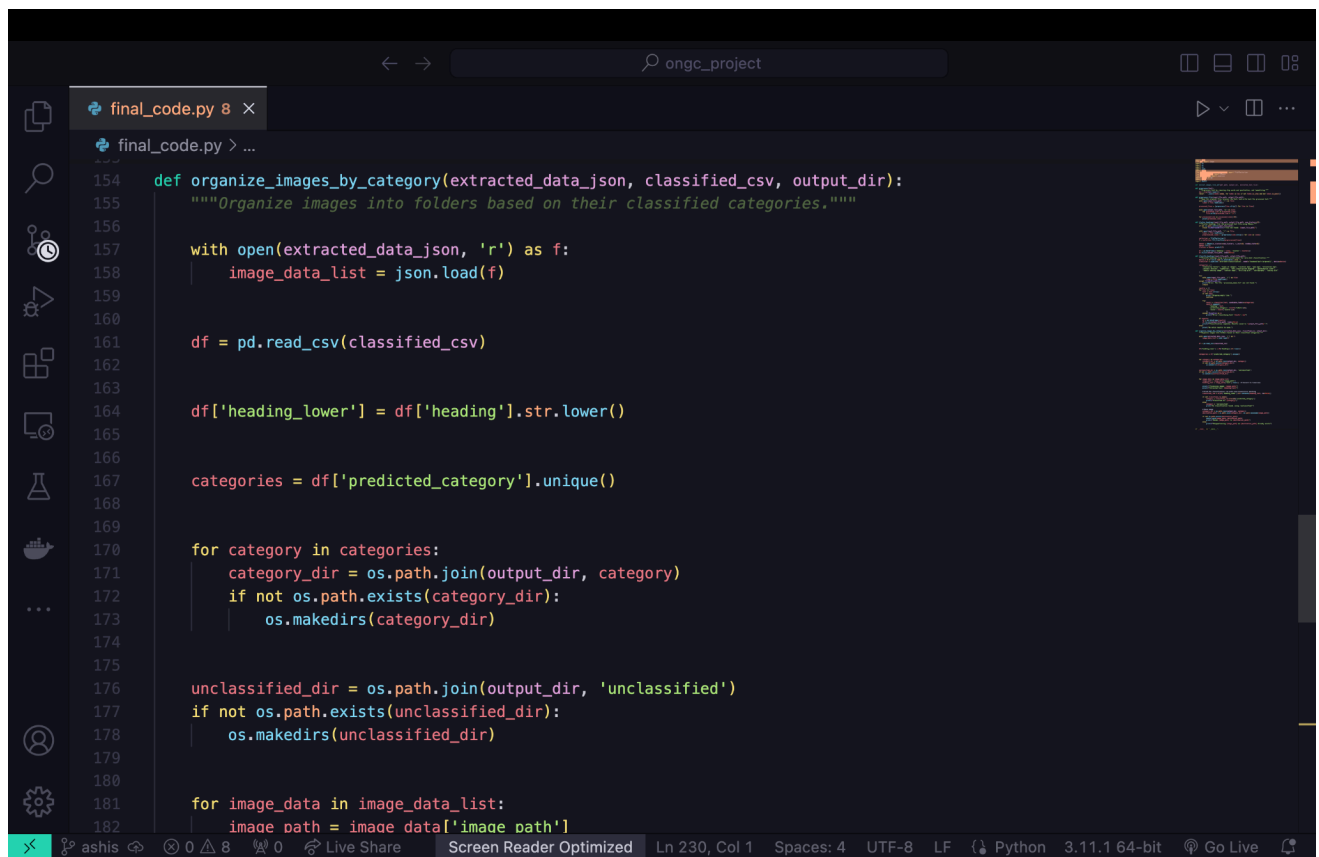
```
final_code.py 8 x
final_code.py > ...
91     print(processed_line)
92
93 def cluster_headings(input_file_path, output_file_path, num_clusters=10):
94     """Cluster headings from the processed text file using KMeans."""
95     if not os.path.isfile(input_file_path):
96         raise FileNotFoundError(f"File not found: {input_file_path}")
97
98     with open(input_file_path, 'r') as file:
99         lines = file.readlines()
100         preprocessed_lines = [preprocess(line.strip()) for line in lines]
101
102     vectorizer = TfidfVectorizer()
103     X = vectorizer.fit_transform(preprocessed_lines)
104
105     kmeans = KMeans(n_clusters=num_clusters, n_init=10, random_state=42)
106     kmeans.fit(X)
107     clusters = kmeans.predict(X)
108
109     df = pd.DataFrame({'heading': lines, 'cluster': clusters})
110     df.to_csv(output_file_path, index=False)
111
112 > def classify_headings(input_file_path, output_file_path):--
153
154 > def organize_images_by_category(extracted_data_json, classified_csv, output_dir):...
```

CLASSIFICATION OF HEADINGS:

```
← → ongo_project
final_code.py 8 X
final_code.py > ...
112 def classify_headings(input_file_path, output_file_path):
113     """Classify headings using a pre-trained model for zero-shot classification."""
114     device = 0 if torch.cuda.is_available() else -1
115     classifier = pipeline('zero-shot-classification', model='facebook/bart-large-mnli', device=device)
116
117     categories = [
118         "similarity factor", "types of images", "location map", "geo map", "structural map",
119         "seismic section", "logmotive", "well construction diagram", "geotechnical order",
120         "remote sensing image", "contour maps", "drilling plot", "cartography", "casing plot"
121     ]
122
123     try:
124         with open(input_file_path, 'r') as file:
125             lines = file.readlines()
126     except FileNotFoundError:
127         print("Error: The file 'processed_names.txt' was not found.")
128         return
129
130     results = []
131     for text in lines:
132         text = text.strip()
133         if not text:
134             print("Skipping empty line.")
135             continue
136
137         try:
138             result = classifier(text, candidate_labels=categories)
139             results.append({
140                 'heading': text,
```

```
← → ongo_project
final_code.py 8 X
final_code.py > ...
112 def classify_headings(input_file_path, output_file_path):
125     lines = file.readlines()
126 except FileNotFoundError:
127     print("Error: The file 'processed_names.txt' was not found.")
128     return
129
130 results = []
131 for text in lines:
132     text = text.strip()
133     if not text:
134         print("Skipping empty line.")
135         continue
136
137     try:
138         result = classifier(text, candidate_labels=categories)
139         results.append({
140             'heading': text,
141             'predicted_category': result['labels'][0],
142             'score': result['scores'][0]
143         })
144     except Exception as e:
145         print(f"Error classifying text '{text}': {e}")
146
147     if results:
148         df = pd.DataFrame(results)
149         df.to_csv(output_file_path, index=False)
150         print(f"Classification complete. Results saved to '{output_file_path}'.")
151     else:
152         print("No valid results to save.")
```

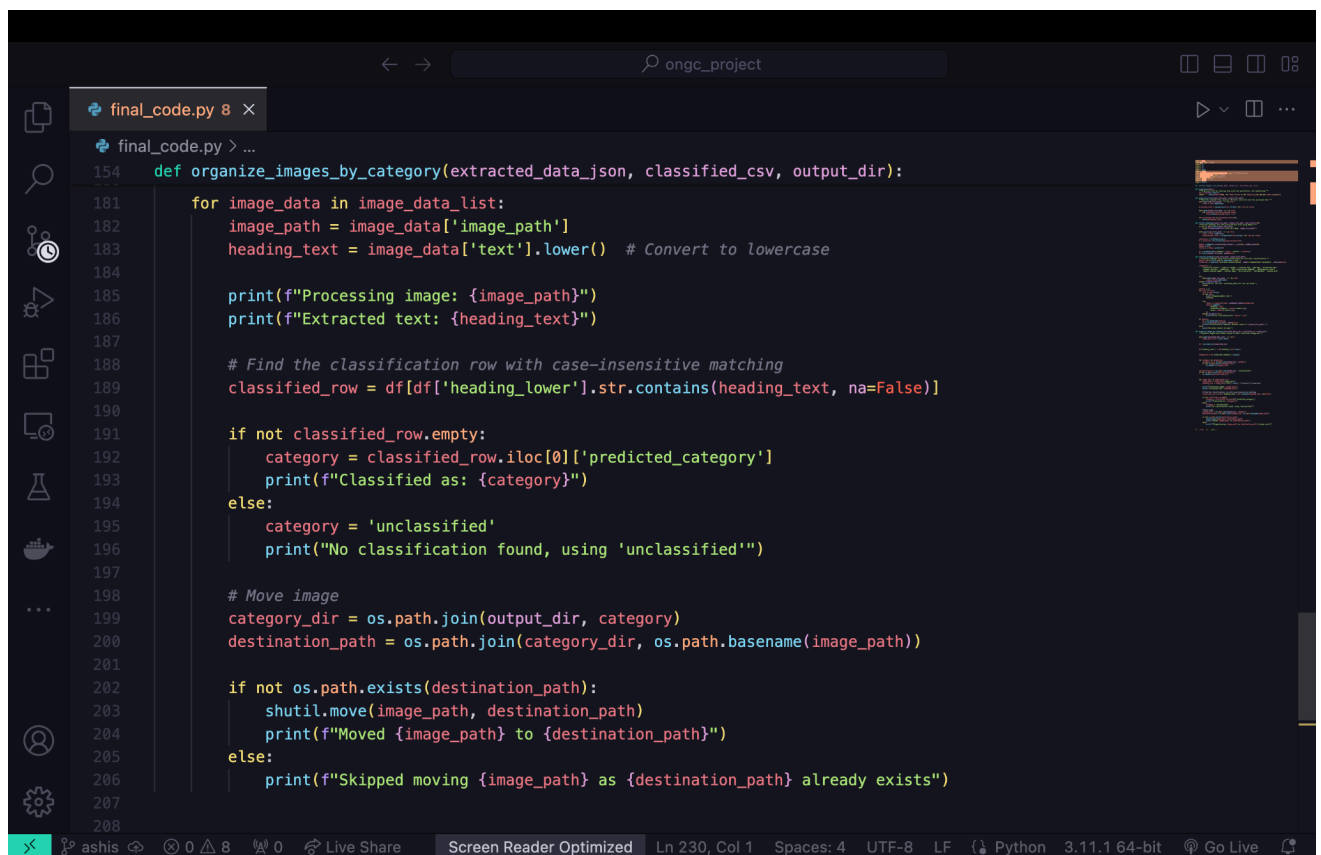
IMAGES ORGANISATION:



The screenshot shows a VS Code editor window with a file named `final_code.py` open. The editor is displaying a Python function `organize_images_by_category` that takes three arguments: `extracted_data_json`, `classified_csv`, and `output_dir`. The function's docstring is `"""Organize images into folders based on their classified categories."""`. The code uses `json` to load the `extracted_data_json` file, `pandas` to read the `classified_csv` file, and `os` to create directories for each category and an unclassified directory. It then iterates over the `image_data_list` and extracts the `image path` for each item.

```
154 def organize_images_by_category(extracted_data_json, classified_csv, output_dir):
155     """Organize images into folders based on their classified categories."""
156
157     with open(extracted_data_json, 'r') as f:
158         image_data_list = json.load(f)
159
160
161     df = pd.read_csv(classified_csv)
162
163
164     df['heading_lower'] = df['heading'].str.lower()
165
166
167     categories = df['predicted_category'].unique()
168
169
170     for category in categories:
171         category_dir = os.path.join(output_dir, category)
172         if not os.path.exists(category_dir):
173             os.makedirs(category_dir)
174
175
176     unclassified_dir = os.path.join(output_dir, 'unclassified')
177     if not os.path.exists(unclassified_dir):
178         os.makedirs(unclassified_dir)
179
180
181     for image_data in image_data_list:
182         image_path = image_data['image path']
```

The status bar at the bottom shows the following information: `ashis`, `0` errors, `8` warnings, `0` info, `Live Share`, `Screen Reader Optimized`, `Ln 230, Col 1`, `Spaces: 4`, `UTF-8`, `LF`, `Python`, `3.11.1 64-bit`, `Go Live`.



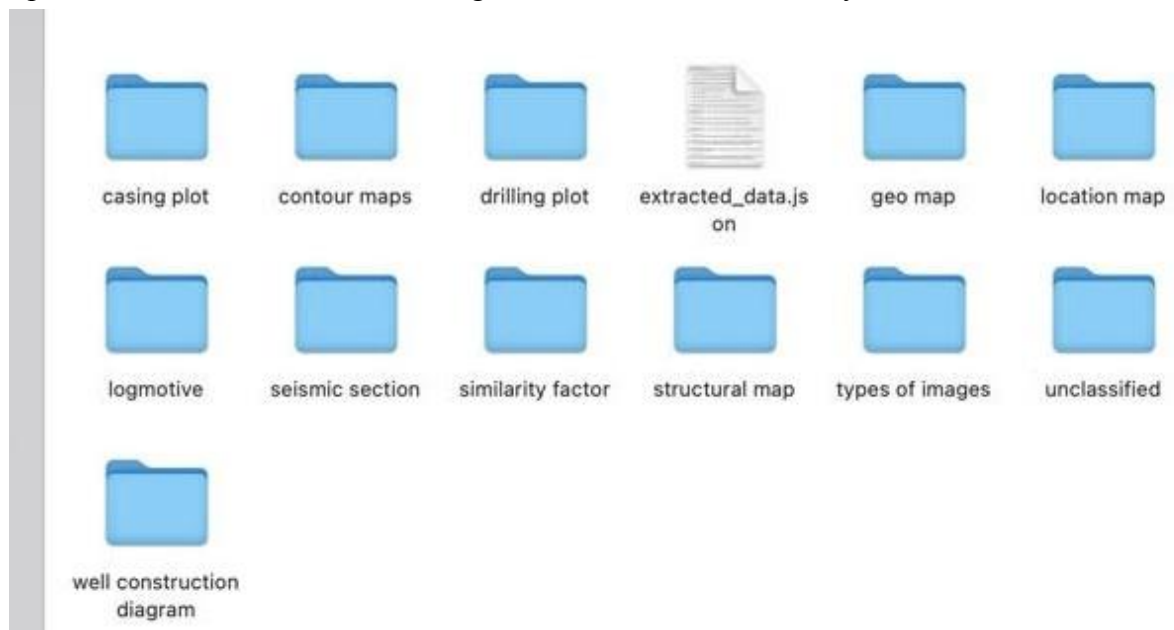
The screenshot shows a code editor with a dark theme. The top bar indicates the project name 'ongo_project'. The editor is open to a file named 'final_code.py'. The code defines a function 'organize_images_by_category' which takes three arguments: 'extracted_data_json', 'classified_csv', and 'output_dir'. The function iterates over 'image_data_list'. For each item, it extracts the 'image_path' and 'text' (converted to lowercase). It then searches for a classification row in 'classified_csv' using 'case-insensitive matching'. If found, it prints the category and moves the image to a directory named after the category. If not found, it prints 'unclassified' and skips moving the image. The status bar at the bottom shows 'ashis', '0 8', 'Live Share', 'Screen Reader Optimized', 'Ln 230, Col 1', 'Spaces: 4', 'UTF-8', 'LF', 'Python 3.11.1 64-bit', and 'Go Live'.

```
154 def organize_images_by_category(extracted_data_json, classified_csv, output_dir):
181     for image_data in image_data_list:
182         image_path = image_data['image_path']
183         heading_text = image_data['text'].lower() # Convert to lowercase
184
185         print(f"Processing image: {image_path}")
186         print(f"Extracted text: {heading_text}")
187
188         # Find the classification row with case-insensitive matching
189         classified_row = df[df['heading_lower'].str.contains(heading_text, na=False)]
190
191         if not classified_row.empty:
192             category = classified_row.iloc[0]['predicted_category']
193             print(f"Classified as: {category}")
194         else:
195             category = 'unclassified'
196             print("No classification found, using 'unclassified'")
197
198         # Move image
199         category_dir = os.path.join(output_dir, category)
200         destination_path = os.path.join(category_dir, os.path.basename(image_path))
201
202         if not os.path.exists(destination_path):
203             shutil.move(image_path, destination_path)
204             print(f"Moved {image_path} to {destination_path}")
205         else:
206             print(f"Skipped moving {image_path} as {destination_path} already exists")
207
208
```

MAIN FUNCTION :

```
final_code.py 8 x
final_code.py > ...
206     print(f"Skipped moving {image_path} as {destination_path} already exists")
207
208
209 if __name__ == "__main__":
210     pdf_path = "well_student.pdf"
211     output_dir = "new5"
212     extracted_text_file = "extracted_names.txt"
213
214     if not os.path.exists(output_dir):
215         os.makedirs(output_dir)
216
217     # Initialize the spaCy model
218     nlp = spacy.load('en_core_web_md')
219
220     extract_images_from_pdf(pdf_path, output_dir, extracted_text_file)
221
222     input_file_path = extracted_text_file
223     output_file_path = 'processed_names.txt'
224     preprocess_file(input_file_path, output_file_path)
225
226     cluster_headings(output_file_path, 'clustered_headings.csv')
227     classify_headings(output_file_path, 'classified_headings.csv')
228
229     organize_images_by_category(os.path.join(output_dir, 'extracted_data.json'), 'classified_headings.csv', outp
230
```

OUTPUT: The result comes in the form of a folder where the predefined categories folders are present and relevant classified images are sorted in the necessary folder.



Name	Date Modified	Size	Kind
▼ casing plot	Today at 2:27 PM	--	Folder
page_21_image_011_m_td_maximum_form...predrill_pore_pressure_plot_ad66adav.png	Today at 2:20 PM	135 KB	PNG image
page_30_imageflt_increase_gradually_43...ase_aafb_agartala_pagfigure51flt_plot.png	Today at 2:20 PM	31 KB	PNG image
page_32_image_completion_report_ad_fig...plot_hole_temperature_reservoir_study.png	Today at 2:20 PM	16 KB	PNG image
page_35_image_completion_report_ad66_adav_figure61_prognose_vs_actual_plot.png	Today at 2:20 PM	161 KB	PNG image
page_40_image_completion_report_ad66_adav_figure_72_horizontal_profile_plot.png	Today at 2:20 PM	53 KB	PNG image
page_65_image_c_isopay_map_ap36_pay.jpeg	Today at 2:20 PM	175 KB	JPEG image
▼ contour maps	Today at 2:27 PM	--	Folder
page_12_image_completion_report_ad6_str...p_close_ap36_pay_sand_seismic_line.jpeg	Today at 2:20 PM	90 KB	JPEG image
page_56_image_time_relief_contour_close_ap36_pay_sand.jpeg	Today at 2:20 PM	121 KB	JPEG image
page_65_image_rms_amplitude_map_rec_r...verlain_time_relief_contour_ap36_pay.jpeg	Today at 2:20 PM	219 KB	JPEG image
▼ drilling plot	Today at 2:27 PM	--	Folder
page_16_image_completion_report_ad66_a...211_total_drilling_time_breakup_chart.png	Today at 2:20 PM	145 KB	PNG image
page_16_image_figure_211_total_drilling_ti...tal_nonproductive_time_breakup_chart.png	Today at 2:20 PM	49 KB	PNG image
page_27_image_completion_report_ad66_a...ure_plot_ad66adav_46_lotpitsit_datum.png	Today at 2:20 PM	271 KB	PNG image
page_29_image_464_graph_pit_8_hole_24...forward_base_aafb_agartala_page_25.png	Today at 2:20 PM	115 KB	PNG image
extracted_data.json	Today at 2:20 PM	11 KB	JSON Document
▼ geo map	Today at 2:27 PM	--	Folder
page_61_image_general_informatio_tripura_geological_map.jpeg	Today at 2:20 PM	247 KB	JPEG image
▼ location map	Today at 2:27 PM	--	Folder
page_11_image_completion_report_adlocation_map_figure11_location_map_adav.jpeg	Today at 2:20 PM	88 KB	JPEG image
page_11_image_figure11_location_map_ada...tellite_view_location_adav_wrt_nearby.jpeg	Today at 2:20 PM	153 KB	JPEG image
page_52_image_plate_2_field_location_map.jpeg	Today at 2:20 PM	87 KB	JPEG image
page_53_image_plate_3_location_map_adav.jpeg	Today at 2:20 PM	182 KB	JPEG image
page_69_image_zm3zq_locationkmap.png	Today at 2:20 PM	953 KB	PNG image
▼ logmotive	Today at 2:27 PM	--	Folder
page_22_image_42_mud_log_datum_421...plot_figure42_composite_pressure_plot.png	Today at 2:20 PM	123 KB	PNG image
page_45_image_completion_report_ad66m...re73_mud_weight_plan_versus_actual.jpeg	Today at 2:20 PM	102 KB	JPEG image
page_59_image_bokabil_formation_logging_service_tripura_asset_agartala.png	Today at 2:20 PM	101 KB	PNG image
▼ seismic section	Today at 2:27 PM	--	Folder
page_12_image_i385_xl1081s.jpeg	Today at 2:20 PM	24 KB	JPEG image
page_12_image_seismic_line_figure14_interpret_seismic_il385.jpeg	Today at 2:20 PM	84 KB	JPEG image
page_12_image_xl1081_pass_adav_s.jpeg	Today at 2:20 PM	73 KB	JPEG image
page_24_image_completion_report_ad66...factor_plot_423_dexponent_sigma_plot.png	Today at 2:20 PM	43 KB	PNG image
page_54_image_seismic_il385_xl1081_pass_location_adav.jpeg	Today at 2:20 PM	228 KB	JPEG image
page_62_image_hrough_adav_ad66xl1081.jpeg	Today at 2:20 PM	169 KB	JPEG image
page_62_image_seismic_section_il385_xl1081_passingil385.jpeg	Today at 2:20 PM	103 KB	JPEG image

new5					Search		
Name		Date Modified	Size	Kind			
page_62_image_hrough_adav_ad66x1081.jpeg		Today at 2:20 PM	169 KB	JPEG image			
page_62_image_seismic_section_il385_xl1081_passingil385.jpeg		Today at 2:20 PM	193 KB	JPEG image			
page_63_image_seismic_section_rc_line_pass_p.jpeg		Today at 2:20 PM	215 KB	JPEG image			
page_66_image_seismic_section_il385_il385.jpeg		Today at 2:20 PM	181 KB	JPEG image			
page_69_image_seismickzlkofk33v_seismickxk33v.png		Today at 2:20 PM	291 KB	PNG image			
page_69_image_seismickzlkofk33v.png		Today at 2:20 PM	330 KB	PNG image			
similarity factor		Today at 2:27 PM	--	Folder			
page_39_image_0_2844_2063_273610_26...ala_pagefigure_71_vertical_profile_plot.png		Today at 2:20 PM	48 KB	PNG image			
page_53_image_page_21.jpeg		Today at 2:20 PM	30 KB	JPEG image			
page_58_image_o_ad66_adav.png		Today at 2:20 PM	23 KB	PNG image			
page_66_image_n_s.jpeg		Today at 2:20 PM	33 KB	JPEG image			
page_66_image_sing_adav_ad66.jpeg		Today at 2:20 PM	164 KB	JPEG image			
page_68_image_rc_line_pass_ro28_ad66.jpeg		Today at 2:20 PM	324 KB	JPEG image			
structural map		Today at 2:27 PM	--	Folder			
page_8_image_completion_report_ad66_a...972_deshpande_et al_1993_mathur2001.jpeg		Today at 2:20 PM	221 KB	JPEG image			
page_52_image_.jpeg		Today at 2:20 PM	19 KB	JPEG image			
page_58_image_ad66_adav.png		Today at 2:20 PM	697 bytes	PNG image			
page_58_image_n.png		Today at 2:20 PM	19 KB	PNG image			
page_59_image_rw_determination_plot_pic...r_bhuban_formation_bokabil_formation.png		Today at 2:20 PM	101 KB	PNG image			
page_62_image_.jpeg		Today at 2:20 PM	54 KB	JPEG image			
page_63_image_.jpeg		Today at 2:20 PM	154 KB	JPEG image			
page_67_image_.jpeg		Today at 2:20 PM	84 KB	JPEG image			
page_68_image_.jpeg		Today at 2:20 PM	84 KB	JPEG image			
page_69_image_.png		Today at 2:20 PM	135 KB	PNG image			
types of images		Today at 2:27 PM	--	Folder			
page_60_image_clay_mineralogy_plot_th_vs_k_crossplot.png		Today at 2:20 PM	33 KB	PNG image			
unclassified		Today at 2:27 PM	--	Folder			
page_25_image_exponent_sigma_value_pl...011_m_figure44_dexponent_sigma_plot.png		Today at 2:20 PM	104 KB	PNG image			
page_28_image_case_size_inch_20_total...m261total_vol_flow_ltr135drilled_dept.png		Today at 2:20 PM	3 KB	PNG image			
page_29_image_case_size_inch13_38total...464_graph_pit_8_hole_2499_m_md_2.png		Today at 2:20 PM	3 KB	PNG image			
page_55_image_time_relief_contour_ap36...ain_rms_attribute_map_ap36_pay_sand.jpeg		Today at 2:20 PM	168 KB	JPEG image			
page_67_image_interpret_rc_line_pass_well_ro28_ad.jpeg		Today at 2:20 PM	324 KB	JPEG image			
well construction diagram		Today at 2:27 PM	--	Folder			
page_33_image_completion_report_ad66...av_figure53_temperature_gradient_plot.png		Today at 2:20 PM	59 KB	PNG image			
page_51_image_completion_report_ad66_figure8_schematic_diagram.png		Today at 2:20 PM	44 KB	PNG image			
page_57_image_construction_diagram_drill...av_log_service_tripura_asset_agartala.jpeg		Today at 2:20 PM	78 KB	JPEG image			

CONCLUSION

In conclusion, the project "Image Extraction and Categorization Using Python and NLP" focuses on developing a robust system for automatically extracting text from images and classifying these images into predefined categories using Natural Language Processing (NLP) techniques. This project leverages the power of Python programming, coupled with advanced NLP frameworks such as spaCy, to process and analyze text data embedded in images." represents a significant step forward in enhancing data extraction and analyses within Oil and Natural Gas Corporation Limited (ONGC). The implementation of automated image extraction, text preprocessing, clustering, and classification workflows underscores our commitment to modernising geological data analysis while prioritising accuracy and efficiency. By allowing youngsters like me to work on these projects and mentoring us for real world tasks ONGC continuous to its values of learning and growing. As we move forward, this project sets a precedent for embracing technology-driven transformations, positioning ONGC at the forefront of organisational efficiency and innovation

BIBLIOGRAPHY

1. www.stackoverflow.com
2. www.geeksforgeeks.com
3. www.github.com
4. www.tutorialspoint.com
5. <https://pymupdf.readthedocs.io/>
6. https://huggingface.co/spacy/en_core_web_sm

